

Produce Your Brand

FROM PRODUCT IDEA TO READY-TO-SEND SUPPLIER REQUEST.

Describe what you want to produce. Produce Your Brand helps you understand how it can be made, find suitable suppliers, and prepare a ready-to-send request.

An AI sourcing assistant for custom physical products

What it does

You describe what you want to produce — 100 yoga mats with a gold logo. It writes the production brief, asks what is missing, proposes a method, finds suppliers who can actually take the job, and drafts the request to send them.

Why this project

Every custom order starts with the same question: *can this be made, and by whom?* Today it is answered by email, over days. That is a reasoning problem, not a search problem — which is exactly what an agent is for.

How it works

One workflow that stops and asks the user before each important step. **The model writes the language; ordinary code decides every fact** — supplier matching contains no model calls at all.

Where it stands

Runs end to end at **\$0.0011 per project**, on 308 automated tests. **[M]** Supplier records are synthetic and labelled as such: the algorithm is real, the partners are not.

The argument in three steps

SITUATION

STEP ONE

Businesses regularly need something custom made — 100 yoga mats with a gold logo, 500 branded bottles before a trade fair. German promotional-products and custom-print production is a €3–4bn industry doing exactly this work. [E]

COMPLICATION

STEP TWO

The hard part is not finding printers. It is establishing whether the job can be made at all: material, technique, minimum quantity, deadline, and whether a supplier will work on goods the customer already owns. Those facts sit in no searchable database — they are discovered one email at a time, over days. Search cannot answer them, and a general chat assistant answers differently each time it is asked.

RESOLUTION

STEP THREE

Produce Your Brand turns the plain-language request into a structured brief and settles feasibility **in code rather than in the model** — matching on real constraints, treating *unknown* as distinct from *no*, and returning a request the user can send. Same input, same answer, every time, at \$0.0011 per project. [M]

SO WHAT Everything after this slide is evidence for the resolution.

The problem is feasibility reasoning, not search

Four sequential unknowns, each of which can invalidate the order — and the buyer discovers them one at a time.

Worked example, used throughout: **100 black yoga mats, PVC, customer already owns them, gold logo, Berlin.**

#	UNKNOWN	WHY IT BLOCKS	COST OF GETTING IT WRONG
1	Is the technique physically possible?	PVC under a laser releases hydrogen chloride. No reputable shop will do it	Weeks lost pursuing an impossible route
2	Who accepts customer-owned goods?	Many producers only work on stock they supply, and most enquiries never think to ask	Quote arrives, then collapses
3	Does the MOQ fit?	100 units sits below many minimums	Silent disqualification
4	Is the deadline feasible?	Lead time plus buffer against the date	Committed delivery that cannot be met

SO WHAT The scarce resource is not a directory of printers. It is structured reasoning over capability constraints.

Neither search nor a general chat assistant closes this gap

CAPABILITY	GOOGLE / FLYERALARM	GENERAL CHAT ASSISTANT	PRODUCE YOUR BRAND
Find printers in a city	✓	✓	✓
Explain a technique	generic	✓	✓ with citation to a specific document
Refuse an unsafe pairing (PVC + laser)	×	inconsistent	✓ grounded in the corpus
Filter by published MOQ	×	×	✓
Filter by customer-owned-goods policy	×	×	✓ including unconfirmed as distinct from no
Deadline feasibility against lead time	×	×	✓
Same answer to a reworded question	✓	×	✓ byte-identical, tie order included
Auditable reason per factor	×	×	✓ six factors, each with a computed reason

SO WHAT The differentiator is proprietary structured capability data, not language ability.

A narrow segment, but one where the unit economics are not the constraint

TAM	German promotional-products and custom-print production	~€3–4bn annual revenue [E]
SAM	Non-standard requests — unusual material, customer-owned goods, odd quantity, tight deadline	~15–25% of enquiries [E]
SOM	Intermediaries who source repeatedly — distributors, merch and event agencies	~3,000–5,000 firms [E]

Who would pay
Not the business ordering once a year. The intermediary sourcing weekly, for whom every hour of email is billable time lost.

What a customer costs to serve
\$0.0011 per project run. A \$249 monthly seat funds roughly 231,000 runs before compute is material. [M][D]

The ceiling on the wedge
4,000 firms at €249 a month is ≈€12m a year fully served. [D] Modest — but at software margin, and nobody serves it today.

All market figures are [E] — order-of-magnitude, from industry reporting and interview inference. They frame the segment; they do not yet size a business case.

The model handles language; code owns every fact

The LLM never touches a number that matters.

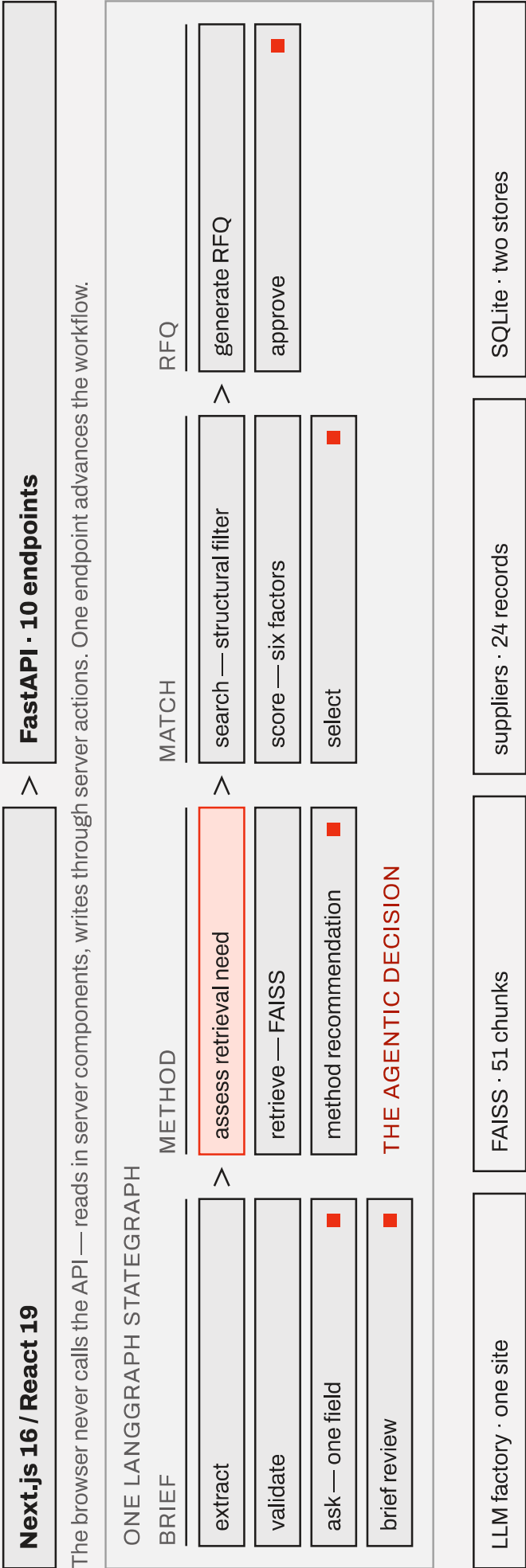
The model owns language

- Extract the brief from free text — natural language into a typed object
- Production method, with retrieval and citations — real technical reasoning
- Match explanation prose, over a score it cannot alter
- RFQ intro and closing — tone only

Code owns every fact

- Which fields are missing — *is this field null* is not a judgement call
- Supplier search — a structural filter that must not depend on phrasing
- Match scoring — reproducible and defensible arithmetic
- RFQ structure — a document shape is a contract, not a generation

One graph, five interrupts, two memory stores



Red squares mark the five human interrupt points; four are approval gates. Python 3.12 · FastAPI · LangGraph · LangChain · OpenAI via OpenRouter · FAISS · fastembed · SQLite · TypeScript · Next.js · Tailwind · Leaflet · Docker · Railway.

The system decides whether to retrieve, and the decision changes the answer

The routing ladder — cheapest signal first

- 1

Deterministic fast path — *which Berlin suppliers do laser engraving?* goes straight to the supplier repository

zero model calls
- 2

Deterministic rule on the brief — material unconfirmed, so feasibility cannot be assumed

zero model calls
- 3

The model — only the genuinely ambiguous middle

1 classifier call

Same request, retrieval off against on [M]

	WITHOUT	GROUNDED
Confidence	medium	high
Constraints returned	1, generic	4, specific
Open questions	type of gold finish	the specific PVC compound
Citations	none	2 documents, named

ANTI-HALLUCINATION retrieval_used and sources are set in code from what was retrieved, never asserted by the model. Retrieved passages are fenced inside a user message like customer text, never the system prompt.

Three datasets, every record carrying its own provenance

The algorithm is real; the partners are not — and the data says so in every row.

DATASET	SCALE [M]	PROVENANCE, STATED IN THE DATA ITSELF
Knowledge base	13 documents · 21,522 bytes · 51 indexed chunks	Curated by hand; a recommendation cites the document by name
Supplier records	24 records	data_source: synthetic in every row; website is null throughout
Offers	4 records	is_demo: true; a validator refuses an offer that is both demo and verified

How it is prepared

- Chunked into a FAISS index — an index file plus JSON, fingerprinted by the effective model name
- Unconfirmed capability stored as null, never defaulted to false
- The set includes suppliers who refuse and suppliers never asked, so both branches are exercised

10 · Produce Your Brand · AI Engineering Capstone · September 2026

What cannot be sourced

Real supplier pricing cannot be reliably scraped — it lives in non-public business logic. Every offer is flagged demo, and substituting real records changes no schema.

Unknown is a third answer, not a soft no

Capability data is incomplete. Treating a missing value as absence of capability is wrong in both directions.

The trap		The resolution — three-way logic, end to end			
TREATMENT	FAILURE	STATE	SCORE	RANKED	SHOWN TO THE BUYER
unknown → no	Viable partners silently discarded; the buyer never learns they existed	Confirmed capable	full points	yes	factor met
unknown → yes	Capability promised that nobody confirmed; the order collapses later	Not asked (null)	partial credit	yes	explicit risk flag
		Explicitly refused (false)	hard gate	no	incompatible — reported separately

IN THE PRODUCT The dataset stores null rather than defaulting to false; the interface renders unknowns as *Not specified*, never as blank; and the dataset deliberately includes suppliers who refuse and suppliers whose policy is unconfirmed, so both branches are exercised.

\$0.0011

Cost of one complete project on gpt-4o-mini — roughly **929 projects per dollar**. Inference is not a cost driver at any plausible volume. [M][D]

Verification designed so a published claim cannot drift from the code

308

automated tests — none calling a live model

19/19

behavioural eval cases, all passing

15

architecture invariants that fail the build

0.65:1

test-to-application code ratio

The eval covers the failure modes that matter

CATEGORY	CASES
Complete brief proceeds without interrogation	1
Incomplete brief asks one field, in published priority order	2–6
Non-blocking gaps do not trigger a question	7–8
Deadline feasibility, incl. past and impossible dates	9–12
null scored differently from false	12, 14, 15
Hard gates: impossibilities excluded, never ranked	16–17
MOQ, and the funnel over the real dataset	18–19

Live end-to-end, verified [M]

- Extraction, then a RAG-grounded method — heat_transfer, high confidence, correct documents cited
 - 3 eligible of 7 candidates, each with a computed breakdown
 - RFQ generated unapproved, completed only on approval. Zero errors
- What the eval does not claim: that the weighting is correct. Whether material compatibility deserves 20 of 100 points is a question for customer evidence, not a test.

Both production defects shared one root cause: a code path with no test

Defect 1 — a 90% match with an expired deadline

Found by hand on the deployed site. A supplier scored 90% while the deadline factor read *the deadline has already passed* — a fatal, request-wide fact diluted into one of six weighted rows.

Root cause. The extraction prompt did not require a year-less date to resolve forward, so *by September 15* could land in the past.

Safety net. A guard nulls any deadline earlier than today, after both extraction and the clarification merge. That branch had no test at all; two now exist.

Defect 2 — an opaque HTTP 500 in production

`generate_image` wrapped only the API call in `try/except` while response parsing ran unguarded, so an unexpected response shape raised a raw `AttributeError` the caller's `except LLMError` could never catch.

The class had zero direct tests, which is precisely how it shipped. **Six now exist, two of them regression tests for the exact shapes that used to crash.**

```
status_code=402  body={"error":{"message":"You requested up to 1024 tokens,
                  but can only afford 881", ...}}
```

DIAGNOSABILITY The failing call now logs the status code and the provider's raw body, which names its own remedy — converting a recurring mystery into a one-line configuration fix.

Ranked by impact on usefulness rather than by effort

#	NEXT	WHY IT RANKS HERE	TYPE
1	Real partner data	24 synthetic records is the honest gap. This is what turns a prototype into something a buyer can act on	Field work
2	Send the RFQ	The document is complete and human-approved; nothing transmits it. Also the precondition for measuring monetisation	Engineering
3	Authentication	Projects are addressable by UUID today — adequate for a single-tenant prototype, not for real users	Engineering
4	Per-field provenance	verified and data_source describe a whole record, not the specific capability a buyer is trusting	Schema
5	Price-aware matching	Offer.price_from drives the best-price view, but cost is not one of the six scored factors	Engineering

Explicitly not built — payments, a supplier portal, more production categories. Each would broaden a product with no paying user yet.
Known limitations: synthetic supplier data · demo-only offers · no authentication · location scored in tiers rather than by distance · no streaming · one light theme.

Questions

The model writes language. Code owns every fact. 308 tests, 19 of 19 eval cases, \$0.0011 per project — and 24 synthetic suppliers still to replace.